

Excel Date & Time Functions

A reference guide: purpose, syntax, examples, business use cases, edge cases, and limitations

1. TODAY()

Purpose: Returns the current system date. Used to build dynamic sheets that auto-update daily, such as age calculators, "days since" trackers, and deadline reminders.

Syntax: `=TODAY()` — no arguments.

Example:

Formula: `=TODAY()` → Output (on 30-Jun-2026): **30-06-2026**

Business Use Cases:

- Calculating outstanding invoice age: `=TODAY() - InvoiceDate`
- Flagging overdue tasks against a deadline column
- Dynamic "as of" date labels on dashboards

Edge Cases: Volatile — recalculates every time the workbook opens or any cell changes, so the value shifts daily without user action. Returns a serial number formatted as a date; if a cell shows a number like 46172 instead of a date, the cell formatting needs fixing.

Limitations: Cannot be "frozen" to a fixed date without converting to a static value (copy → paste special → values). Not suitable for audit trails that need a fixed creation date.

2. NOW()

Purpose: Returns the current date and time together. Used for timestamping actions or calculating elapsed time down to the minute or second.

Syntax: `=NOW()` — no arguments.

Example:

Formula: `=NOW()` → Output: **30-06-2026 14:35**

Business Use Cases:

- Logging when a record was last updated
- Calculating turnaround time in customer support tickets
- Time-stamping form submissions

Edge Cases: Volatile and recalculates constantly — two cells with `=NOW()` may show slightly different times if entered seconds apart. Time depends on the system clock, which can be inaccurate if not synced.

Limitations: Needs to be pasted as a static value to "lock" a timestamp. Precision is only to the second; not suitable for millisecond-level logging.

3. DATE()

Purpose: Constructs a valid Excel date from separate year, month, and day values. Essential when dates are scattered across columns or built from calculations.

Syntax & Arguments: `=DATE(year, month, day)`

- **year** - a 4-digit year (or cell reference)
- **month** - 1 to 12 (values outside this range roll over into adjacent years)
- **day** - 1 to 31 (values outside this range roll over into adjacent months)

Example:

Formula: `=DATE(2026, 6, 30)` → Output: **30-06-2026**

Business Use Cases:

- Combining separate Year/Month/Day columns from a database export into one date field
- Generating fiscal year start dates dynamically, e.g. `=DATE(YEAR(TODAY()), 4, 1)` for an April fiscal year

Edge Cases: `=DATE(2026, 13, 1)` doesn't error — it rolls forward to January 2027 since Excel treats overflow as arithmetic, not validation. `=DATE(2026, 0, 1)` returns December 2025.

Limitations: Doesn't validate that the result is a "sensible" business date — invalid-looking inputs silently produce a different valid date instead of an error, which can hide data entry mistakes.

4-6. YEAR(), MONTH(), DAY()

Purpose: Extract the year, month, or day number from a date value. Used to group, filter, or compare data at a coarser time granularity.

Syntax & Arguments:

- `=YEAR(serial_number)` → returns 1900-9999
- `=MONTH(serial_number)` → returns 1-12
- `=DAY(serial_number)` → returns 1-31

Example:

For date 15-08-2026 in cell A1:

`=YEAR(A1)` → 2026 | `=MONTH(A1)` → 8 | `=DAY(A1)` → 15

Business Use Cases:

- Building pivot-style summaries (sales by month/year) without a pivot table
- Creating helper columns to group transactions by year for YoY analysis
- Extracting day-of-month to identify month-end transactions

Edge Cases: If the input cell contains text that looks like a date but isn't a true date value, these functions return a #VALUE! error. They still work on plain numbers since Excel reads the underlying serial number.

Limitations: Return numbers, not names — MONTH() gives 8, not "August" — needs TEXT() for labels. No built-in fiscal-year logic.

7. WEEKDAY()

Purpose: Returns which day of the week a date falls on, as a number. Used to identify weekends, schedule recurring events, or build calendars.

Syntax & Arguments: `=WEEKDAY(serial_number, [return_type])`

- **serial_number** – the date
- **return_type** – optional code: 1 (default) Sunday=1...Saturday=7; 2 Monday=1...Sunday=7; 3 Monday=0...Sunday=6

Example:

Formula: `=WEEKDAY(DATE(2026,6,30),2)` → Output: 2 (Tuesday, since type 2 makes Monday=1)

Business Use Cases:

- Flagging weekend dates in a delivery schedule for exclusion
- Calculating weekend overtime pay
- Building "is this a business day" checks alongside IF()

Edge Cases: Forgetting the return_type argument is a common error — the default Sunday=1 numbering can misclassify weekends if the analyst assumes Monday=1.

Limitations: Doesn't know about public holidays — only distinguishes weekday vs weekend, so must be paired with WORKDAY/NETWORKDAYS for true business-day logic.

8. WEEKNUM()

Purpose: Returns which week number of the year a date falls in. Used for weekly reporting cycles and trend analysis.

Syntax & Arguments: `=WEEKNUM(serial_number, [return_type])`

- **return_type** – optional; defines which day starts the week (1 = Sunday-start, default; 2 = Monday-start; 21 = ISO week numbering)

Example:

Formula: `=WEEKNUM(DATE(2026,1,15),2)` → Output: 3

Business Use Cases:

- Weekly sales or production tracking
- Aligning data to a company's weekly reporting calendar
- Grouping data in pivot tables by week instead of by date

Edge Cases: Standard WEEKNUM (return_type 1 or 2) doesn't follow ISO 8601 numbering used internationally; for ISO weeks, return_type 21 must be used, otherwise results won't match calendars used across Europe and most businesses.

Limitations: Week 1 definitions vary by convention, so results can disagree with a printed calendar unless the correct return_type is chosen deliberately.

9. DATEDIF()

Purpose: Calculates the difference between two dates in years, months, or days. Commonly used for age, tenure, or duration calculations.

Syntax & Arguments: `=DATEDIF(start_date, end_date, unit)`

- **start_date / end_date** – the two dates being compared (start must be earlier)
- **unit** – "Y" (full years), "M" (full months), "D" (days), "MD" (days ignoring months/years), "YM" (months ignoring years), "YD" (days ignoring years)

Example:

Formula: `=DATEDIF(DATE(2020,3,15), DATE(2026,6,30), "Y")` → Output: **6 years**

Business Use Cases:

- Calculating employee tenure for HR reports
- Customer age verification
- Contract duration tracking
- Loan term calculations

Edge Cases: If start_date is later than end_date, it returns a #NUM! error. The "MD" unit can give unreliable results in certain date combinations due to a known Excel calculation quirk.

Limitations: It's an undocumented legacy function — not listed in Excel's function library or autocomplete help, though it still works. No live error-checking hints, making mistakes harder to debug.

10. EDATE()

Purpose: Adds or subtracts a specified number of months from a date, returning a new date with the same day-of-month (adjusted if needed). Used for recurring monthly schedules.

Syntax & Arguments: `=EDATE(start_date, months)`

- **start_date** – the base date
- **months** – positive number to move forward, negative to move backward

Example:

Formula: `=EDATE(DATE(2026,1,31), 1)` → Output: **28-02-2026** (auto-adjusted since February has no 31st)

Business Use Cases:

- Calculating subscription renewal dates
- Loan / EMI due dates
- Warranty expiry dates
- Calculating "6 months from hire date" for probation reviews

Edge Cases: When the start day doesn't exist in the target month (e.g. the 31st), EDATE automatically shifts to that month's last valid day rather than erroring.

Limitations: Only handles whole-month shifts — can't add a mix of months and days in one call; needs combining with other functions for more complex offsets.

11. EOMONTH()

Purpose: Returns the last day of a month, a set number of months before or after a given date. Used for month-end reporting and billing cycles.

Syntax & Arguments: `=EOMONTH(start_date, months)`

- **start_date** – the base date
- **months** – 0 for current month-end, positive/negative to shift forward/backward

Example:

Formula: `=EOMONTH(DATE(2026,6,15), 0)` → Output: **30-06-2026**

Business Use Cases:

- Generating month-end closing dates for financial statements
- Calculating the last day for return/refund policies
- Scheduling recurring month-end reports automatically

Edge Cases: `=EOMONTH(start_date, -1)` is a common pattern to get the previous month-end for prior-period comparisons — but the sign is easy to get backward.

Limitations: Returns only month-end dates — cannot return, say, the 15th of a future month; that requires DATE() or EDATE() instead.

12. WORKDAY()

Purpose: Calculates a date a specified number of working days before or after a start date, automatically skipping weekends (and optionally holidays). Used for project deadlines and delivery date planning.

Syntax & Arguments: `=WORKDAY(start_date, days, [holidays])`

- **start_date** – base date
- **days** – number of working days to add (negative to go backward)
- **holidays** – optional range of dates to also exclude

Example:

Formula: `=WORKDAY(DATE(2026,6,30), 5)` → Returns the date 5 working days later, skipping Saturdays/Sundays

Business Use Cases:

- Calculating promised delivery dates excluding weekends
- Setting project milestone deadlines
- Calculating "payment due in 10 business days"

Edge Cases: Without a holiday list, only Saturday/Sunday are skipped — public holidays still count as working days unless explicitly listed. The holidays argument must be real date values, not text.

Limitations: Assumes a fixed Saturday–Sunday weekend; doesn't support different weekend patterns (e.g. Friday–Saturday in some regions) — that requires `WORKDAY.INTL()`.

13. NETWORKDAYS()

Purpose: Counts the number of whole working days between two dates, excluding weekends and optional holidays. Used to measure project duration or staffing needs.

Syntax & Arguments: `=NETWORKDAYS(start_date, end_date, [holidays])`

- **start_date / end_date** – the date range (both inclusive)
- **holidays** – optional range of dates to exclude

Example:

Formula: `=NETWORKDAYS(DATE(2026,6,1), DATE(2026,6,30))` → Counts working days in June 2026, excluding weekends

Business Use Cases:

- Calculating billable working days for contractors
- Estimating delivery timelines
- Calculating SLA (service-level agreement) compliance windows
- Payroll day counts

Edge Cases: Both `start_date` and `end_date` are counted as part of the range (inclusive), which can cause off-by-one errors if exclusive counting is assumed. Like `WORKDAY()`, only skips Saturday/Sunday by default.

Limitations: Fixed weekend assumption limits use in regions with different weekend days — `NETWORKDAYS.INTL()` is needed there. Doesn't account for partial working days (e.g. half-day holidays).

14. TIME()

Purpose: Constructs a valid time value from separate hour, minute, and second components. Used when time data is split across columns or calculated.

Syntax & Arguments: `=TIME(hour, minute, second)`

- **hour** – 0 to 23
- **minute** – 0 to 59
- **second** – 0 to 59

Example:

Formula: `=TIME(14,30,0)` → Output: **2:30 PM**

Business Use Cases:

- Building shift schedules from separate hour/minute inputs
- Calculating elapsed time for call-center logs
- Constructing appointment time slots

Edge Cases: Values exceeding the normal range wrap around — `=TIME(25,0,0)` doesn't error, it rolls over to 1:00 AM, since it's treated as a fraction of a 24-hour day, similar to `DATE()`'s rollover behavior.

Limitations: Returns only a time fraction (0 to <1), with no date component — combining a `TIME()` with a calendar date requires adding it to a `DATE()` value separately.

15. HOUR(), MINUTE(), SECOND()

Purpose: Extract the hour, minute, or second component from a time or datetime value. Used to analyze time-of-day patterns.

Syntax & Arguments:

- `=HOUR(serial_number)` → 0-23
- `=MINUTE(serial_number)` → 0-59
- `=SECOND(serial_number)` → 0-59

Example:

For timestamp 14:35:20 in cell A1:

`=HOUR(A1)` → **14** | `=MINUTE(A1)` → **35** | `=SECOND(A1)` → **20**

Business Use Cases:

- Identifying peak call/website-traffic hours from log timestamps
- Calculating shift start/end compliance
- Segmenting transactions into time-of-day buckets (morning/afternoon/evening) for sales analysis

Edge Cases: If the source cell only contains a date (no time component), these functions correctly return 0 for hour/minute/second rather than erroring, which can mask missing time data if not checked.

Limitations: All three return whole numbers within their range only — no built-in way to get fractional seconds or 12-hour AM/PM labels directly; that needs TEXT() formatting on top.